

Interaction Manager Candidate 기반 리팩토링

1. 문제 상황

기존 CInteraction_Manager는 Player 주변에 실제 후보가 하나뿐이어도 Scene의 전체 Interaction Object를 매 Frame 순회했다. Before Characterization에서는 Registry N=25, InRange K=1인데도 IsInteratable()이 프레임당 25회 호출됐다. Manager Update는 600 Frame 기준 평균 10.233 μ s, 최대 388.600 μ s였다.

불필요한 연산의 원인은 “전체 Object 관리”와 “Player Target 탐색”을 같은 Registry loop가 담당한 데 있었다.

2. 기존 구조

전체 Registry N

- 매 Frame 전체 IsInteratable
- Range lifecycle
- strict '>' Priority
- CurrentTarget
- 다시 전체 Registry를 순회해 Interaction 정리

확인된 구조적 위험은 다음과 같았다.

- 실제 주변 Object 수 K와 무관한 전체 Object 반복 탐색
- Carry early return에 의존한 암묵적 CurrentTarget 유지
- 일부 파생 클래스에 흩어진 unregister와 raw pointer lifetime
- PLAYER/MONSTER가 공유하는 TriggerBox collision 상태
- Query 함수 안에서 실행되는 비관련 Gameplay Side Effect

3. AI 사전 분석

AI는 Candidate container 선택, 실행 중 Target 분리 여부, TriggerBox와 Manager의 의존 방향, Collision callback 반영 timing, Registry 유지 여부를 선택지로 제시했다. 또한 Before Characterization과 raw pointer lifetime 검증이 필요하다고 분석했다.

이 분석은 최종 결정이 아니라 개발자 검토 입력으로 사용됐다. 개발자는 실제 Update 순서, strict > 순서 의존성, Carry Anim-Notify의 CurrentTarget 사용, 기존 Scene/Lifetime 코드를 확인한 뒤 규칙을 확정했다.

4. 개발자의 설계 판단

항목	AI 제안	개발자 검토	최종 결정
Candidate Container	vector 또는 hash 계열	등록/제거보다 순회가 빈번하고 동일 Priority 순서 보존 필요	std::vector, stable erase, 중복 방지
CurrentTarget / ActiveInteractionTarget	실행 중 Pointer 분리 가능	Pointer 동기화 책임과 기존 AnimNotify 동일성 위험	별도 Pointer 없이 Interaction 중 CurrentTarget Lock
Candidate 등록 책임	TriggerBox의 Manager 직접 요청도 가능	Trigger는 충돌 사실, Interaction 정책은 Owner 책임	TriggerBox → CInteractable → Manager
Pending 반영	callback 즉시 변경 또는 Pending	Manager Update 뒤 Collision Late가 실행되므로 다음 Update 시작 반영이 기존 timing 수준	최종 상태를 병합한 Pending Add/Remove
Registry	Candidate-only 또는 병행	최적화 목적은 Registry 삭제가 아니라 탐색 범위 축소	Registry는 Life-time/Scene/Script용으로 유지

Decision 2와 3은 AI 선택지를 그대로 적용하지 않았다. 개발자는 별도 Target Pointer를 추가하지 않았고 TriggerBox의 Manager 직접 의존도 허용하지 않았다.

5. 개선 구조

Registry = 전체 Interaction Object 관리
Candidate = 현재 Player Trigger 주변 후보
Pending = Collision Frame에서 발생한 최종 Candidate 상태
CurrentTarget = 선택된 Target, Interaction 중 lifecycle Lock

PLAYER Trigger Enter / Exit
→ CIIInteractable 공통 진입점
→ Pending Add / Remove
→ 다음 Manager Update 시작 시 Apply
→ Candidate K만 IsInteractable / Priority 검사
→ CurrentTarget

Candidate는 연속 메모리의 std::vector이고 동일 우선순위에서는 먼저 들어온 후보가 strict > 비교로 유지된다. Registry loop는 Clear()와 Change_Scene() 같은 관리 경로에는 남지만 매 Frame Target 탐색에는 사용되지 않는다.

6. CurrentTarget Lock 구조

Rock Candidate 진입
→ CurrentTarget = Rock
→ A / Interaction 시작
→ m_bPrevInteracting=true, CurrentTarget Lock
→ Trigger Disable / Pending Remove / Candidate 제거
→ CurrentTarget Rock 유지
→ Carry Animation / PlayerOnCarry / Attach
→ Interaction 종료 시 Lock 해제

Candidate와 CurrentTarget의 의미를 분리했기 때문에 범위를 떠난 실행 중 Object를 유지하기 위해 별도의 ActiveInteractionTarget이 필요하지 않다. 실제 Object Destroy에서는 PurgeInteractable()이 Lock보다 우선해 pointer를 제거한다.

개발자가 직접 확인한 Carry Range, UI, Carry State, PlayerOnCarry, Attach, Throw는 [PASS]였다. CurrentTarget identity와 기타 세부 단계는 별도 검증 결과가 없어 [NOT VERIFIED]로 유지한다.

7. Lifetime 안정성 개선

CIIInteractable은 Candidate Add/Remove와 Manager Purge의 공통 정책을 제공한다. 프로젝트의 두 공통 interactable Owner 계층인 CInteractionObject와 CNPC가 Free 시 ReleaseInteractionRegistration()을 호출한다.

Purge는 다음 위치의 동일 Pointer를 idempotent하게 정리한다.

- 전체 Registry
- Candidate vector
- Pending Request vector
- CurrentTarget / PreviousTarget
- Debug 시작 Target
- 활성 Interaction/Range 상태

Disable은 Destroy와 구분한다. TriggerBox Disable은 Candidate Remove를 요청하지만 Registry는 유지한다. TriggerBox Part가 파괴될 때는 이미 Purge된 Owner를 Pending에 다시 넣지 않도록 내부 overlap만 비운다. Scene Clear는 Registry를 이용해 lifecycle callback을 정리한 뒤 Registry/Candidate/Pending/Target을 함께 비운다.

8. Before / After 결과

After benchmark는 개발자 Gameplay 검증 완료 후 기존 F10 checkpoint와 F8/F7 Debug 계층만 사용해 Town의 Rock Range에서 600 Frame을 측정했다. 측정 중 Town을 유지하기 위해 개발자 요청대로 DebugR → Boss 전환만 주석 처리했으며

신규 profiler/instrumentation은 추가하지 않았다.

항목	Before	After	변화
Registry N	25	25	유지
Candidate K	1 (기존 InRange)	1.000	안정 600 Frame 동안 1
IsInteratable Calls / Frame	25.000	1.000	-96.000%, N회에서 K회로 전환
Priority Compare / Frame	0.000	0.000	단일 Candidate이라 동일
Manager Update Avg	10.233 μ s	8.945 μ s	-12.587% (-1.288 μ s)
Manager Update Max	388.600 μ s	85.800 μ s	-77.921% (-302.800 μ s)
Measurement Frames	600	600	동일

After의 latest/min/max/avg K가 모두 1이고 IsInteratable=1.000/frame이었다. Registry 25개를 유지하면서 Target Selection 호출 수가 Candidate 1개와 일치했으므로 N→K 전환이 측정값으로 확인됐다. Debug 실행 시간에는 scheduling noise가 있을 수 있어 호출 구조와 함께 해석한다.

9. Gameplay 검증

개발자가 직접 보고한 항목만 PASS로 반영했다.

항목	상태
Interaction Range	[PASS]
Interaction UI	[PASS]
CurrentTarget	[NOT VERIFIED]
A Input	[NOT VERIFIED]
Carry State	[PASS]
Carry Animation	[NOT VERIFIED]
PlayerOnCarry	[PASS]
Target Identity	[NOT VERIFIED]
Attach	[PASS]
Holding	[NOT VERIFIED]
Throw	[PASS]
Normal State Return	[NOT VERIFIED]
Player JackyBall Grab	[PASS]
Monster JackyBall Grab	[PASS]
Player/Monster 동일 Ball Gameplay	[NOT VERIFIED]

10. 리팩토링 중 발견한 숨은 의존성과 설계 판단

전체 Registry 순회를 제거하는 과정에서 Query 함수 내부에 존재하던 비관련 Gameplay Side Effect를 발견했다. CInteraction_JackyBall::IsInteratable()은 Player 가능 여부를 묻는 동시에 CM_Jacky::Grab(this)까지 실행하고 있었다.

JackyBall은 Monster 전용도 Player 전용도 아니며, Player와 Monster 모두가 상호작용하는 하나의 Object였다. 따라서 JackyBall 자체 역할을 둘로 나누거나 Player Candidate에서 제외하지 않았다. 하나의 Object가 두 Gameplay 경로를 유지하도록 하면서 실행 책임만 분리했다.

Player overlap → Candidate → IsInteratable Query → CurrentTarget → Player Grab

Monster overlap → JackyBall Update → 기존 조건 → CM_Jacky::Grab

전체 순회를 다시 유지하는 대신 Side Effect를 실제 Gameplay Owner인 JackyBall Update로 옮겼다. 이로써 Candidate 기반 Target 탐색 구조를 유지하면서 Monster Gameplay가 실행될 수 있는 구조가 됐다. 개발자 검증에서 Player Grab과 Monster Grab은 각각 [PASS]였다. 동시 ownership 경쟁은 [NOT VERIFIED]다.

11. AI 활용 과정

AI

- 기존 코드 분석
- 위험 요소와 설계 선택지 제안

개발자

- 실제 코드와 Update 순서 확인
- Before Baseline 측정
- Trade-off 검토
- AI 제안 채택 / 수정 / 기각
- 5개 최종 설계 규칙 확정

AI

- 확정 설계 기준 구현
- 정적 검토와 Client Debug x64 Build
- 기존 계측 기능으로 동일 Rock Range 600 Frame After 측정

개발자

- Carry 및 JackyBall Gameplay 직접 검증

AI의 분석을 입력으로 사용하되 시스템 특성과 실제 측정치를 근거로 개발자가 최종 설계 판단을 수행했다.

12. 최종 성과

- 전체 Registry의 매 Frame Target 탐색 순회 제거: [IMPLEMENTED]
- Candidate K 기반 탐색: [IMPLEMENTED]
- Pending 최종 상태 병합과 stable vector: [IMPLEMENTED]
- 별도 Active Pointer 없는 CurrentTarget Lock: [IMPLEMENTED]
- 공통 lifetime Purge와 Player/Monster overlap 분리: [IMPLEMENTED]
- Client Debug | x64 Build 및 Link: [PASS]
- 개발자가 명시한 Carry 6개 항목과 JackyBall Player/Monster Grab: [PASS]
- N→K 호출 구조: [MEASURED] (25.000 → 1.000 calls/frame, -96.000%)
- Manager 평균/최대: [MEASURED] (8.945 μ s / 85.800 μ s, 600 Frame)

13. 남은 개선 가능성

- CurrentTarget identity, Carry Animation/Holding/정상 복귀처럼 명시적으로 보고되지 않은 세부 항목을 필요 시 별도 확인한다.
- Player와 Monster가 동시에 동일 JackyBall ownership을 경쟁하는 동작을 필요 시 확인한다.
- Room cache 비활성화/재활성화 후 Trigger collision 재참여를 확인해야 한다.

Collision System 전체 재설계, 새로운 Player/Monster Priority 규칙, NPC/Quest/Inventory 구조 변경은 이번 범위에서 의도적으로 제외했다.